

Convertidores en JSF



Convertidores en JavaServer Faces (JSF)

Los **convertidores** en JSF son herramientas que transforman los datos de entrada de un formulario web (normalmente cadenas de texto) a tipos específicos de Java (como fechas, números, etc.) y viceversa. Esto permite que los valores ingresados por el usuario sean procesados adecuadamente en el backend.

1. Concepto Básico

Un convertidor en JSF es una clase o una funcionalidad integrada que toma un valor en formato texto (por ejemplo, una fecha o un número) y lo convierte a un tipo de dato específico de Java. Esto es especialmente útil cuando trabajamos con tipos como `Date`, `Integer`, `Double`, entre otros.

JSF proporciona convertidores predefinidos, como:

- `f:convertDateTime`: Para manejar fechas y horas.
- `f:convertNumber`: Para convertir cadenas a números.

También puedes crear **convertidores personalizados** cuando las necesidades del proyecto lo requieran.

2. Sintaxis Básica

Los convertidores en JSF se integran en los componentes de entrada como `<h:inputText>` mediante la etiqueta `<f:convertXXX>`. Aquí hay un ejemplo básico:

```
<h:inputText id="fechaNacimiento" required="true" value="#{candidato.fechaNacimiento}">
    <f:convertDateTime pattern="dd/MM/yyyy" />
</h:inputText>
```

En este caso:

- **f:convertDateTime** convierte el texto ingresado por el usuario a un objeto `java.util.Date`.
- **pattern="dd/MM/yyyy"** define el formato de la fecha esperado.

3. Ejemplo Completo

Candidato.java (Managed Bean)

Esta clase contiene los atributos que serán manipulados en el formulario, como `nombre`, `apellido`, `salarioDeseado` y `fechaNacimiento`.

```
package beans.model;

import jakarta.enterprise.context.SessionScoped;
import jakarta.inject.Named;
import java.io.Serializable;
import java.util.Date;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

@Named("candidato")
@SessionScoped
public class Candidato implements Serializable {

    private final Logger log = LoggerFactory.getLogger(Candidato.class);
    private String nombre;
    private String apellido;
    private int salarioDeseado;
    private Date fechaNacimiento;

    public Candidato() {
        log.info("Creando el objeto Candidato");
        this.setNombre("Introduce tu nombre");
    }
}
```

```
// Métodos getter y setter
public String getNombre() {
    return nombre;
}

public void setNombre(String nombre) {
    this.nombre = nombre;
    log.info("Modificando la propiedad de nombre:" + this.nombre);
}

public String getApellido() {
    return apellido;
}

public void setApellido(String apellido) {
    this.apellido = apellido;
    log.info("Modificando la propiedad de apellido: " + this.apellido);
}

public int getSalarioDeseado() {
    return salarioDeseado;
}

public void setSalarioDeseado(int salarioDeseado) {
    this.salarioDeseado = salarioDeseado;
    log.info("Modificando la propiedad de sueldoDeseado: " + this.salarioDeseado);
}

public Date getFechaNacimiento() {
    return fechaNacimiento;
}

public void setFechaNacimiento(Date fechaNacimiento) {
    this.fechaNacimiento = fechaNacimiento;
}
}
```

Explicación del Código

- **@Named:** Permite que este bean sea accesible en el archivo `xhtml` mediante la expresión `#{candidato}`.
- **@SessionScoped:** Define que el bean tendrá un ciclo de vida a nivel de la sesión HTTP.
- **Logger:** Registra mensajes para depuración y seguimiento.

index.xhtml (Formulario de entrada)

Este archivo contiene un formulario con campos para capturar información sobre el candidato.

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:h="jakarta.faces.html"
      xmlns:f="jakarta.faces.core">
<h:head>
  <title>Bolsa de Trabajo V2</title>
</h:head>
<h:body>
  <h:form>
    <h:messages globalOnly="true" >/h:messages>
    <table>
      <tr>
        <td><h:outputLabel for="nombre" value="Nombre:">/h:outputLabel</td>
        <td><h:inputText id="nombre" required="true"
          value="#{candidato.nombre}">/h:inputText</td>
        <td><h:message for="nombre">/h:message</td>
      </tr>
      <tr>
        <td><h:outputLabel for="apellido" value="Apellido:">/h:outputLabel</td>
        <td><h:inputText id="apellido" required="true"
          value="#{candidato.apellido}">/h:inputText</td>
        <td><h:message for="apellido">/h:message</td>
      </tr>
      <tr>
        <td><h:outputLabel for="salarioDeseado" value="Salario Deseado:">/h:outputLabel</td>
        <td><h:inputText id="salarioDeseado" required="true"
          value="#{candidato.salarioDeseado}">
          <f:validateLongRange minimum="500" maximum="5000"/>
        </h:inputText>
        <td><h:message for="salarioDeseado">/h:message</td>
      </tr>
      <tr>
        <td><h:outputLabel for="fechaNacimiento"
          value="Fecha de Nacimiento:">/h:outputLabel</td>
        <td><h:inputText id="fechaNacimiento" required="true"
          value="#{candidato.fechaNacimiento}">
          <f:convertDateTime pattern="dd/MM/yyyy"/>
        </h:inputText>
        <td></td>
      </tr>
```

```
        <td><h:message for="fechaNacimiento"></h:message></td>
    </tr>
</table>
<h:commandButton action="#{vacanteForm.enviar}" value="Enviar" />
</h:form>
</h:body>
</html>
```

Explicación del Código

1. **Nombre y Apellido:**
 - Los campos `nombre` y `apellido` son obligatorios (`required="true"`).
 - Se utilizan etiquetas `<h:message>` para mostrar mensajes de validación específicos.
2. **Salario Deseado:**
 - Incluye un validador `<f:validateLongRange>` que restringe los valores entre 500 y 5000.
3. **Fecha de Nacimiento:**
 - Usa el convertidor `<f:convertDateTime>` para asegurarse de que la fecha ingresada siga el formato `dd/MM/yyyy`.

4. Temas Importantes

Validadores

Los validadores trabajan junto con los convertidores para garantizar que los datos sean válidos antes de ser procesados. En el ejemplo, `f:validateLongRange` asegura que el salario esté en el rango permitido.

Manejo de Errores

Cuando los datos ingresados no cumplen con los requisitos, JSF muestra mensajes de error específicos mediante `<h:message>` o `<h:messages>`.

5. Resultado

Bolsa de Trabajo V2

localhost:8080/ConvertidoresJSF/index.xhtml

Nombre: Carlos

Apellido: Perez

Salario Deseado: 500

Fecha de Nacimiento: 02/04/

j_idt5:fechaNacimiento: '02/04/' no se ha podido reconocer como fecha. Ejemplo: 20/12/2024

Enviar

6. Conclusión

Los convertidores en JSF simplifican la manipulación de tipos de datos complejos en formularios web. Al utilizarlos junto con validadores y un diseño claro, puedes crear aplicaciones robustas y fáciles de usar.

Sigue explorando las posibilidades de JSF para implementar funcionalidades avanzadas como convertidores personalizados o validaciones específicas.

Saludos!

Ing. Ubaldo Acosta

Fundador de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)